

Tháng 3 2026

ADAPTER

Trình bày bởi Nhóm 5

NỘI DUNG

01. Vấn đề cần giải quyết

02. Định nghĩa adapter

03. Ý tưởng cốt lõi

04. Thành phần chính

05. UML

06. Luồng hoạt động

07. Đánh giá

08. So sánh

VẤN ĐỀ CẦN GIẢI QUYẾT

Đặt tình huống thực tế:

- App đang dùng interface ITarget nhưng thư viện bên thứ ba lại có interface khác.
- Hệ thống cũ trả dữ liệu XML, module mới chỉ xử lý JSON.
- Ứng dụng cần gọi nhiều cổng thanh toán nhưng mỗi bên đặt tên hàm khác nhau.
- Không muốn sửa code client đang ổn định chỉ vì một class/service mới không tương thích

Làm sao dùng được một object có interface không khớp mà không sửa client code và không đung nhiều vào class cũ?

ĐỊNH NGHĨA ADAPTER PATTERN

Adapter Pattern cho phép các object có interface không tương thích có thể làm việc với nhau bằng cách đặt một lớp trung gian để chuyển đổi lời gọi từ interface mà client mong muốn sang interface thật của object được tái sử dụng.

Adapter là một design pattern thuộc nhóm Structural Pattern vì nó tập trung vào cách ghép nối cấu trúc giữa các lớp/object

Ý TƯỞNG CỐT LỖI

Ý tưởng chính:

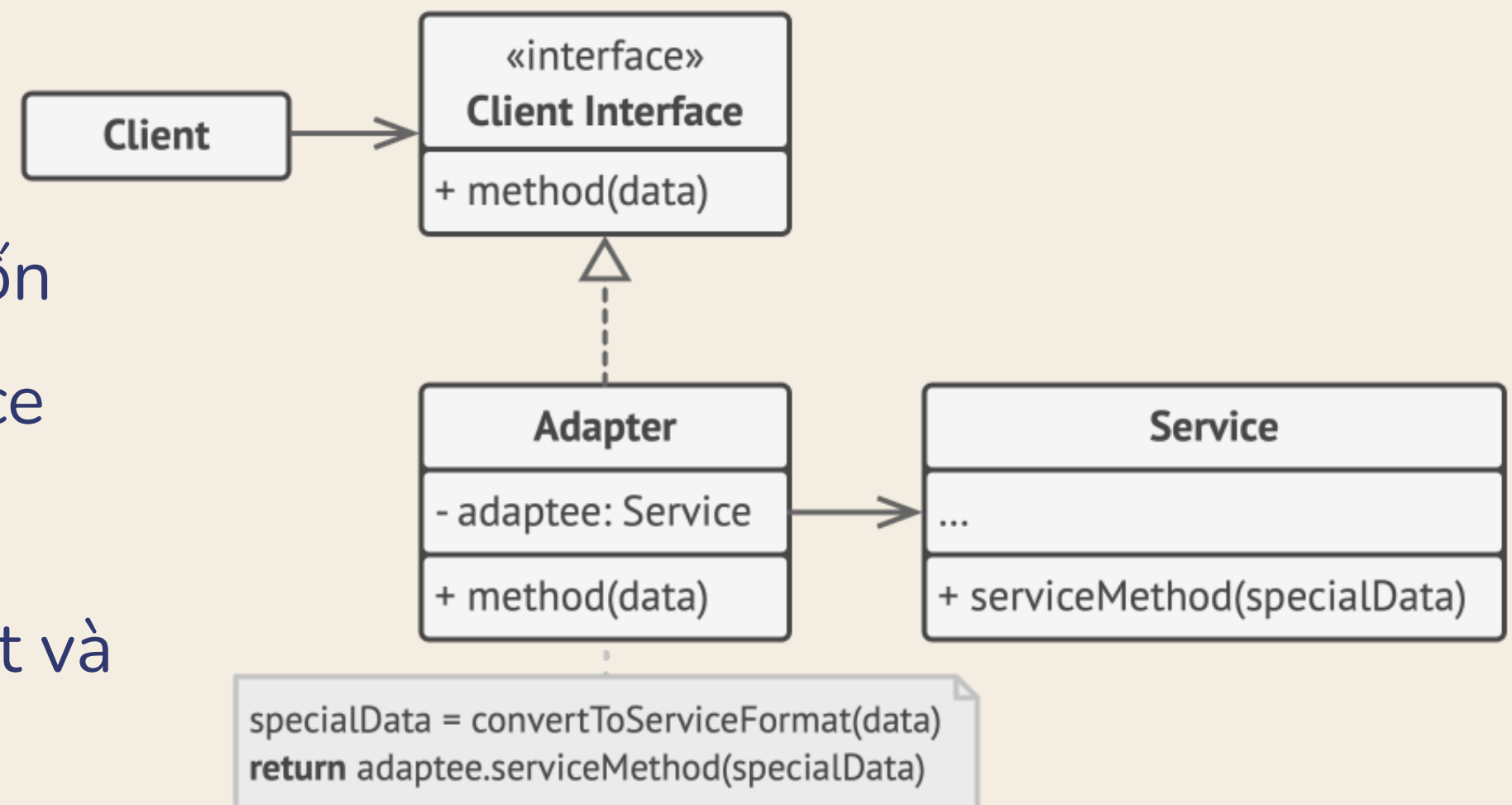
- Bọc một object có interface không tương thích bên trong một object mới, rồi object mới đó cung cấp interface mà client mong muốn.

Adapter thường làm 3 việc:

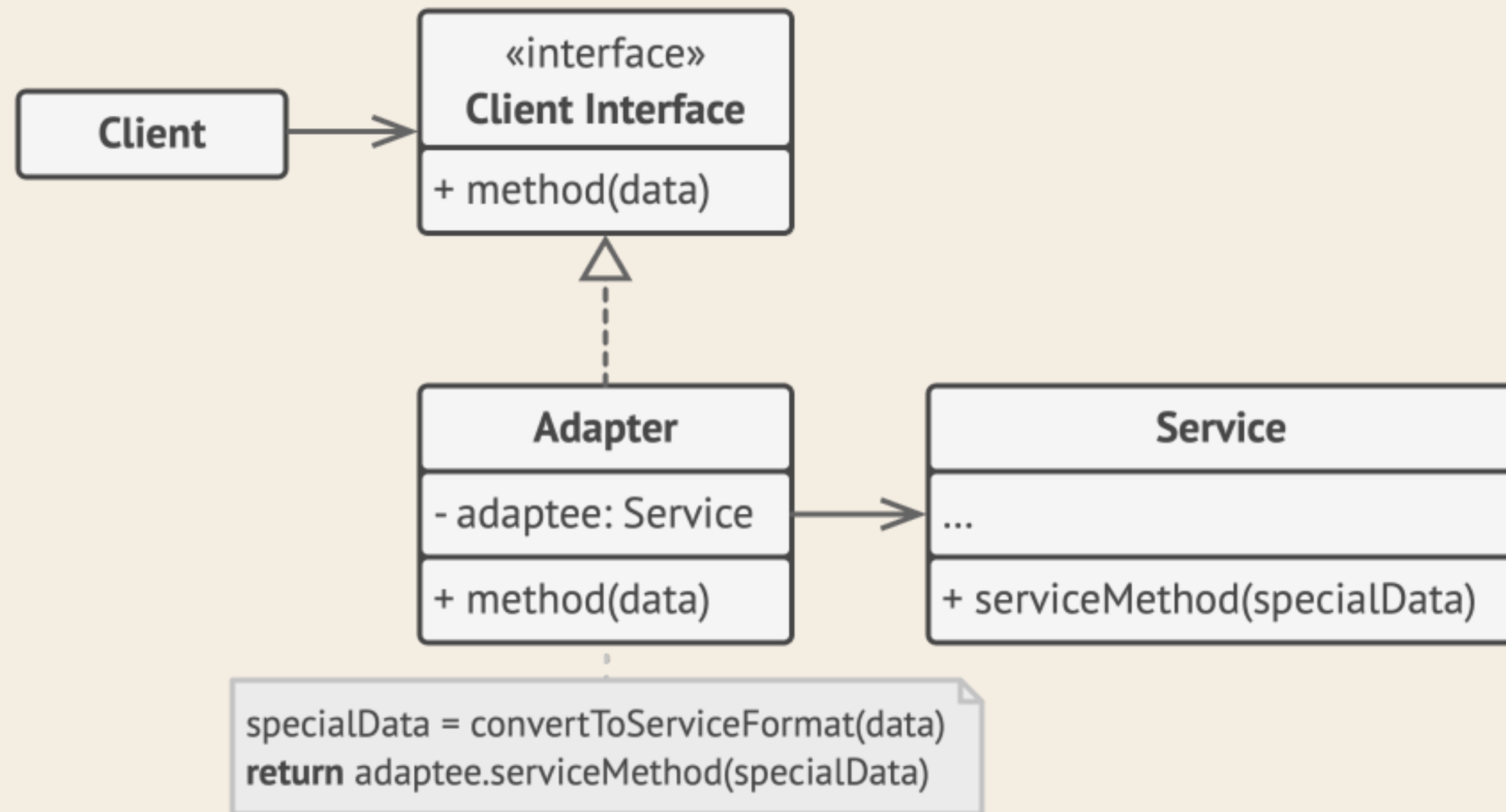
- Nhận request từ client theo interface chuẩn.
- Chuyển đổi request đó sang dạng mà object cũ hiểu được.
- Gọi phương thức thật sự của object cũ.

THÀNH PHẦN CHÍNH

1. Client: Đối tượng sử dụng interface chuẩn
2. Target: Interface mà client mong muốn
3. Adaptee: Class có sẵn nhưng interface không tương thích
4. Adapter: Class trung gian giúp Target và Adaptee làm việc với nhau



UML



LUỒNG HOẠT ĐỘNG

Luồng hoạt động của Adapter Pattern:

1. Client gọi phương thức thông qua interface Target.
2. Adapter nhận lời gọi đó.
3. Adapter chuyển đổi dữ liệu hoặc lời gọi sang dạng phù hợp với Adaptee.
4. Adapter gọi phương thức thật sự của Adaptee.
5. Adaptee xử lý logic.
6. Kết quả được trả ngược lại cho Client nếu có.

ĐÁNH GIÁ

Ưu điểm	Nhược điểm
Giảm phụ thuộc giữa hệ thống và class bên ngoài	Tăng số lượng class
Tái sử dụng được class cũ	Không phù hợp nếu interface khác
Tuân thủ Open/Closed Principle	
Tách biệt logic chuyển đổi	

SO SÁNH

Adapter & Facade

Tiêu chí	Adapter	Facade
Mục đích chính	Chuyển đổi interface không tương thích	Đơn giản hóa một hệ thống phức tạp
Vấn đề giải quyết	Client không dùng được class có sẵn vì khác interface	Client không muốn làm việc trực tiếp với nhiều class phức tạp
Có thay đổi interface không?	Có, chuyển interface này sang interface khác	Có thể có, nhưng mục tiêu là đơn giản hóa
Thường bọc	Một class hoặc một service không tương thích	Nhiều class/subsystem
Ví dụ	Bọc StripeService thành IPaymentGateway	Tạo OrderFacade gọi Inventory, Payment, Shipping

SO SÁNH

Adapter & Decorator

Tiêu chí	Adapter	Decorator
Mục đích chính	Làm interface không tương thích trở nên tương thích	Thêm chức năng mới cho object
Interface sau khi bọc	Thường khác với Adaptee	Thường giống object gốc
Có thay đổi hành vi không?	Có thể chuyển đổi lời gọi	Có, bằng cách bổ sung hành vi
Tập trung vào	Tương thích interface	Mở rộng chức năng
Ví dụ	StripeAdapter biến StripeService thành IPaymentGateway	LoggingPaymentGateway thêm logging trước/sau khi thanh toán

SO SÁNH

Adapter vs Bridge

Tiêu chí	Adapter	Bridge
Nhóm pattern	Structural	Structural
Mục đích	Kết nối các interface không tương thích	Tách abstraction khỏi implementation
Thời điểm dùng	Thường dùng sau khi đã có class không tương thích	Thường thiết kế từ đầu để dễ mở rộng
Quan hệ chính	Adapter bọc Adaptee	Abstraction giữ tham chiếu đến Implementor
Vấn đề chính	“Tôi có class này nhưng interface không khớp”	“Tôi muốn abstraction và implementation phát triển độc lập”

THÀNH VIÊN

Trương Hoàng Phúc

Code

Lê Minh Phát

Trình bày Adapter

Ya Đạt

Trình bày Builder

Tháng 3 2026

THANK YOU

for Listening

Trình bày bởi Nhóm 5